

9.3 Conditional Compilation

[This section corresponds to K&R Sec. 4.11.3]

The last preprocessor directive we're going to look at is `#ifdef`. If you have the sequence

```
#ifdef name
program text
#else
more program text
#endif
```

in your program, the code that gets compiled depends on whether a preprocessor macro by that *name* is defined or not. If it is (that is, if there has been a `#define` line for a macro called *name*), then `program text` is compiled and `more program text` is ignored. If the macro is not defined, `more program text` is compiled and `program text` is ignored. This looks a lot like an `if` statement, but it behaves completely differently: an `if` statement controls which statements of your program are executed at run time, but `#ifdef` controls which parts of your program actually get compiled.

Just as for the `if` statement, the `#else` in an `#ifdef` is optional. There is a companion directive `#ifndef`, which compiles code if the macro is *not* defined (although the `else` clause of an `#ifndef` directive will then be compiled if the macro *is* defined). There is also an `#if` directive which compiles code depending on whether a compile-time expression is true or false. (The expressions which are allowed in an `#if` directive are somewhat restricted, however, so we won't talk much about `#if` here.)

Conditional compilation is useful in two general classes of situations:

- You are trying to write a portable program, but the way you do something is different depending on what compiler, operating system, or computer you're using. You place different versions of your code, one for each situation, between suitable `#ifdef` directives, and when you compile the program in a particular environment, you arrange to have the macro names defined which select the variants you need in that environment. (For this reason, compilers usually have ways of letting you define macros from the invocation command line or in a configuration file, and many also predefine certain macro names related to the operating system, processor, or compiler in use. That way, you don't have to change the code to change the `#define` lines each time you compile it in a different environment.)

For example, in ANSI C, the function to delete a file is `remove`. On older Unix systems, however, the function was called `unlink`. So if `filename` is a variable containing the name of a file you want to delete, and if you want to be able to compile the program under these older Unix systems, you might write

```
#ifdef unix
    unlink(filename);
#else
    remove(filename);
#endif
```

Then, you could place the line

```
#define unix
```

at the top of the file when compiling under an old Unix system. (Since all you're using the macro `unix` for is to control the `#ifdef`, you don't need to give it any replacement text at all.

Any definition for a macro, even if the replacement text is empty, causes an `#ifdef` to succeed.)

(In fact, in this example, you wouldn't even need to define the macro `unix` at all, because C compilers on old Unix systems tend to predefine it for you, precisely so you can make tests like these.)

- You want to compile several different versions of your program, with different features present in the different versions. You bracket the code for each feature with `#ifdef` directives, and (as for the previous case) arrange to have the right macros defined or not to build the version you want to build at any given time. This way, you can build the several different versions from the same source code. (One common example is whether you turn debugging statements on or off. You can bracket each debugging printout with `#ifdef DEBUG` and `#endif`, and then turn on debugging only when you need it.)

For example, you might use lines like this:

```
#ifdef DEBUG
printf("x is %d\n", x);
#endif
```

to print out the value of the variable `x` at some point in your program to see if it's what you expect. To enable debugging printouts, you insert the line

```
#define DEBUG
```

at the top of the file, and to turn them off, you delete that line, but the debugging printouts quietly remain in your code, temporarily deactivated, but ready to reactivate if you find yourself needing them again later. (Also, instead of inserting and deleting the `#define` line, you might use a compiler flag such as `-DDEBUG` to define the macro `DEBUG` from the compiler invocation line.)

Conditional compilation can be very handy, but it can also get out of hand. When large chunks of the program are completely different depending on, say, what operating system the program is being compiled for, it's often better to place the different versions in separate source files, and then only use one of the files (corresponding to one of the versions) to build the program on any given system. Also, if you are using an ANSI Standard compiler and you are writing ANSI-compatible code, you usually won't need so much conditional compilation, because the Standard specifies exactly how the compiler must do certain things, and exactly which library functions it must provide, so you don't have to work so hard to accommodate the old variations among compilers and libraries.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)